Memory System

Table of Contents

- 1. Overview and Design Philosophy
  - 1.1 Core Design Principles
  - 1.2 Principle Two: Four-Point Timestamp Anchoring
  - 1.3 Principle Three: Side Query
  - 1.4 Principle Four: TRUSTING_RECALL_SECTION
  - 1.5 Principle Five: Strict Scope Isolation
  - 1.6 extractMemories Cooperative Mutex Design
  - 1.7 Why Plain Text over Database (Speculation)
- 2. File Structure Overview
  - 2.1 Module Directory Structure
- 3. Memory Type System Explained
  - 3.1 Four Type Definitions
  - 3.2 Scope and Save Specifications per Type
  - 3.3 Frontmatter Format Specification
  - 3.4 Memory Drift Warning
  - 3.5 Parsing Functions
- 4. Storage Mechanism: File System Architecture
  - 4.1 Directory Structure (Three-Layer Scope)
  - 4.2 MEMORY.md Index File (Entrypoint)
  - 4.3 Auto Memory Directory Path Resolution
  - 4.4 Team Memory Path
- 5. Core Data Structures
  - 5.1 MemoryHeader (Scan Result)
  - 5.2 RelevantMemory (Retrieval Result)
  - 5.3 MemoryFileInfo (Complete Memory File)
  - 5.4 SyncState (Team Memory Sync State)
  - 5.5 SyncResult Response Types
  - 5.6 SessionMemoryConfig (Session Memory Configuration)
- 6. Retrieval and Search Algorithm
  - 6.1 Memory File Scanning (scanMemoryFiles)
  - 6.2 AI Relevance Retrieval (findRelevantMemories)
  - 6.3 Conditional Rules
- 7. Memory Staleness Mechanism
  - 7.1 Algorithm Implementation
  - 7.2 Trigger Conditions
  - 7.3 Design Intent
- 8. Session Memory System
  - 8.1 System Positioning

    - 8.2 Extraction Trigger Conditions
    - 8.3 Session Memory Template Structure
    - 8.4 Extraction Execution Flow
    - 8.5 Session Memory Compaction
- 9. Team Memory Cloud Sync System
    - 9.1 API Endpoints
    - 9.2 Sync State Machine
    - 9.3 Full Sync Algorithm
    - 9.4 Hash Algorithm
    - 9.5 File System Watcher
    - 9.6 Secret Scanner
- 10. Agent Memory System
    - 10.1 Three Scopes
    - 10.2 Snapshot System
- 11. @include Directive Explained
    - 11.1 Syntax
    - 11.2 Parsing Flow (claudemd.ts)
    - 11.3 Supported File Types
    - 11.4 Conditional @include (paths field)
- 12. Memory Loading Order and Priority
    - 12.1 Four-Level Loading Order
    - 12.2 Content Merge Strategy
- 13. Security Mechanisms
    - 13.1 Path Traversal Protection
    - 13.2 Config Injection Protection
    - 13.3 Sensitive Information Protection
    - 13.4 File Permissions
- 14. Complete Function Index
    - 14.1 Auto Memory Module (memdir/)
    - 14.2 Session Memory Module (services/SessionMemory/)
    - 14.3 Team Memory Sync Module (services/teamMemorySync/)
    - 14.4 Agent Memory Module (tools/AgentTool/)
    - 14.5 Core Utility Module (utils/)
    - 14.6 Auto-Dream Module (services/autoDream/)
    - 14.7 Memory Extraction Module (services/extractMemories/)
- 15. System Architecture Overview
    - 15.1 Architecture Diagram
    - 15.2 Background Process Diagram
- 16. Algorithm Complexity Analysis
    - 16.1 Operation Complexity Table
- 17. Auto-Dream System (Automatic Memory Consolidation)
    - 17.1 System Positioning

    - 17.2 Lock File Mechanism (consolidationLock.ts)
    - 17.3 Four-Phase Consolidation Prompt (consolidationPrompt.ts)
    - 17.4 Tool Constraints
    - 17.5 DreamTask UI Integration
    - 17.6 GrowthBook Configuration and Toggle
    - 17.7 Relationship with /dream Skill
- 18. ExtractMemories System (End-of-Session Memory Extraction)
    - 18.1 System Positioning
    - 18.2 Forked Agent Mode
    - 18.3 Closure State and Trailing Run
    - 18.4 Submission Pipeline
    - 18.5 Extraction Prompt Templates
- 19. DreamTask (Dream Task UI Integration)
    - 19.1 Design Background
    - 19.2 Task State Structure
    - 19.3 Message Listening (makeDreamProgressWatcher)
    - 19.4 Kill Task
- 20. Daily Log Mode (Assistant Mode / KAIROS)
    - 20.1 Background
    - 20.2 Differences from Standard Mode
- 21. Startup Chain and Module Initialization
    - 21.1 backgroundHousekeeping.ts
    - 21.2 stopHooks.ts Trigger Chain
    - 21.3 Drain on Shutdown
- Appendix A: Zod Schema Definitions
    - A.1 Schema Definitions
- Appendix B: Global State Overview
    - B.1 Global State per Module

---

1. Overview and Design Philosophy

Claude Code's memory system is a **layered, file-based, long-term persistent** architecture, using plain text (Markdown + YAML frontmatter) as the storage medium instead of a database.

PS:
YAML frontmatter = the memory's "index card"

- name, description, type — used to find this memory and determine relevance
  Markdown body = the memory's "main content"
- the actual information to remember

Relationship: frontmatter is metadata attached to the top of a Markdown file; both are written in the same .md file

	YAML frontmatter	Markdown body
目的	检索、分类、相关性判断	存储实际内容
格式	结构化键值对	自然语言文本
使用时机	加载时快速扫描、 relevance 过滤	命中介入时才作为上下文读入

1.1 Core Design Principles

Principle One: Do not save derivable information -> i.e., information that can be directly re-derived.

The following are explicitly defined as "should NOT be stored in memory":

- Code patterns, conventions, architecture (derivable from current project state)
- Git history (use `git log` / `git blame`)
- Debug solutions (the fix code itself is the authoritative source)
- Content already in CLAUDE.md
- Ephemeral task details

> **eval-validated supplement in source code**: In `memoryTypes.ts`, the `WHAT_NOT_TO_SAVE_SECTION` has a final paragraph:
>
> ```
> // H2: explicit-save gate. Eval-validated (memory-prompt-iteration case 3,
> // 0/2 → 3/3): prevents "save this week's PR list" → activity-log noise.
> 'These exclusions apply even when the user explicitly asks you to save.
>  If they ask you to save a PR list or activity summary,
>  ask what was *surprising* or *non-obvious* about it — that is the part worth keeping.'
> ```
>
> This is an **eval-driven design decision**: without this rule, the model would directly execute "user requested save," causing activity-log noise. Core insight: **user request to save ≠ should save** — only the insight *behind* what to save is worth keeping.

1.2 Principle Two: Four-Point Timestamp Anchoring

Each memory carries an `mtimeMs` (modification time) in the file, and the system calculates freshness from it. Memories older than 1 day trigger a "staleness warning," requiring the user to verify the current code state.

```
// memoryAge.ts
export function memoryAgeDays(mtimeMs: number): number {
  return Math.max(0, Math.floor((Date.now() - mtimeMs) / 86_400_000))
}

export function memoryFreshnessText(mtimeMs: number): string {
  const d = memoryAgeDays(mtimeMs)
  if (d <= 1) return ''                          // today/yesterday → no notice
  if (d <= 7) return `This memory is ${d} days old.`
  if (d <= 30) return `This memory is ${d} days old. Verify before citing.`
  // > 30 days → stale reasoning text
}
```

> **Design rationale** (from `memoryAge.ts` comments):
> *"Models are poor at date arithmetic — a raw ISO timestamp doesn't trigger*

> *staleness reasoning the way '47 days ago' does."*
>
> Language models have poor arithmetic reasoning on ISO timestamps. Providing "47 days ago" directly triggers the model's awareness of memory staleness more effectively than "2026-02-12T...". Additionally, the warning text deliberately emphasizes "point-in-time observation" — **a memory is a snapshot of state at a specific point in time, not current state** — which is especially important for memories containing file:line references (those references may have long since失效).

1.3 Principle Three: Side Query

Relevance retrieval uses **Sonnet model for independent queries**, consuming no main conversation context tokens and not affecting the main conversation's reasoning flow.

```
// SELECT_MEMORIES_SYSTEM_PROMPT key design snippet
const SELECT_MEMORIES_SYSTEM_PROMPT = `...
- If a list of recently-used tools is provided, do not select memories that
  are usage reference or API documentation for those tools
  (Claude Code is already exercising them).
  DO still select memories containing warnings, gotchas, or known issues
  about those tools — active use is exactly when those matter.`
```

There is a precise semantic distinction here: **don't select tool usage documentation** (already in the conversation), but **do select tool "warnings/gotchas/known issues"** — during active use, warning information is most useful. Through the `recentTools` parameter, the selector excludes those tools' API reference docs while preserving warning-type memories in active usage scenarios.

> **Telemetry hook** (`findRelevantMemories.ts` , controlled by `feature('MEMORY_SHAPE_TELEMETRY')`): Model selection results and scan results are sent to `logMemoryRecallShape` , recording metrics like `selection_rate` . **Data is still sent when the selection set is empty** — the denominator (`memories.length`) always exists, so it can distinguish "retrieval ran but nothing was selected" from "retrieval never ran" — two completely different product signals.

> **PS: Is memory retrieval asynchronous? Could there be timing issues when injecting into context?**
>
> Answer: This prefetch pattern makes a pragmatic tradeoff between "not blocking" and "trying to catch this round" — using polling instead of blocking wait, allowing prefetch to start later than the API call, but doing its best to complete within the current round (**extremely likely to miss before the API request is sent, but trying to catch before the LLM's streaming output in the main flow**). To strictly guarantee "retrieval must be synchronous," prefetch would need to be changed to await before the for-await — but that introduces an extra RTT delay.

1.4 Principle Four: TRUSTING_RECALL_SECTION — Trust Hierarchy After Memory Recall

This is an **eval-validated design** in `memoryTypes.ts` . The core idea: **finding a memory ≠ being able to trust its content**.

```
// memoryTypes.ts
export const TRUSTING_RECALL_SECTION: readonly string[] = [
```

```
  '## Before recommending from memory',
  'A memory that names a specific function, file, or flag is a claim that ' +
    'it existed *when the memory was written*. It may have been renamed, ' +
    'removed, or never merged. Before recommending it:',
  '- If the memory names a file path: check the file exists.',
  '- If the memory names a function or flag: grep for it.',
  '- If the user is about to act on your recommendation, verify first.',
  '"The memory says X exists" is not the same as "X exists now."',
]
```

1.5 Principle Five: Strict Scope Isolation

The four memory types each have implicit default scopes (private or team), and config injection paths have security restrictions to prevent malicious project configs from writing memories to sensitive directories.

1.6 extractMemories Cooperative Mutex Design — Main Agent and Forked Agent Division

The core design concept of `extractMemories.ts`: **not "two systems competing," but "main agent writes first, forked agent fills gaps."**

```
// extractMemories.ts
/**
 * Returns true if any assistant message after the cursor UUID contains a
 * Write/Edit tool_use block targeting an auto-memory path.
 *
 * The main agent's prompt has full save instructions — when it writes
 * memories, the forked extraction is redundant. runExtraction skips the
 * agent and advances the cursor past this range, making the main agent
 * and the background agent mutually exclusive per turn.
 */
function hasMemoryWritesSince(
  messages: Message[],
  sinceUuid: string | undefined,
): boolean {
  // ...checks if main agent has written to auto-memory path
}
```

Cooperation flow:

1. Main agent **actively writes memories** during conversation (guided by full system prompt) → forked agent detects this, **skips extraction but advances cursor** → main agent wins.
2. Main agent **did not write memories** during conversation → forked agent starts, uses independent extraction prompt to analyze recent conversation and supplement memories.

> **Why this design?** The main agent's system prompt has complete memory-saving instructions; its written memories are "intentional" (based on current conversation semantics). The forked agent is "indiscriminate scanning" (structured extraction based on last N messages). Both have advantages: **main agent writes insights** (why to do something), **forked agent fills in facts** (what happened). Combined they form a complete memory.

Throttle + Trailing Run double insurance:

```
// extractMemories.ts
// Extraction runs once every N eligible turns (tengu_bramble_lintel, default 1)
turnsSinceLastExtraction++
if (turnsSinceLastExtraction < getFeatureValue_CACHED('tengu_bramble_lintel', 1)) {
  return  // skip this turn
}

// If call arrives while extraction is running, stash the context
if (inProgress) {
  pendingContext = { context, appendSystemMessage }  // trailing run
  return
}

// trailing run: handle new messages accumulated between two calls
const trailing = pendingContext
if (trailing) {
  await runExtraction({ context: trailing.context, isTrailingRun: true })
}
```

- **Trailing Run design**: During main extraction, the user may have sent several more messages. Stash the latest context, run the current extraction, then immediately run a trailing with new messages — ensuring nothing is missed.
- **Throttle design** (`tengu_bramble_lintel`): Prevents triggering extraction every turn; default value is 1 (may trigger every turn, but constrained by other conditions).

5-turn hard cap: Extraction agent is force-terminated after at most 5 turns.

> Source comment: `"Well-behaved extractions complete in 2-4 turns (read → write). A hard cap prevents verification rabbit-holes from burning turns."`
>
> Prevents agent from going down "read → verify → read again → verify again" rabbit holes, consuming too many tokens. In practice, 2-4 turns are enough for one effective extraction.

"Single-pass read-then-sort" optimization for memory scanning (`scanMemoryFiles.ts`):

```
// memoryScan.ts
/**
 * Single-pass: readFileInRange stats internally and returns mtimeMs, so we
 * read-then-sort rather than stat-sort-read. For the common case (N ≤ 200)
 * this halves syscalls vs a separate stat round; for large N we read a few
 * extra small files but still avoid the double-stat on the surviving 200.
 */
```

- Common case (N ≤ 200): only reads first 30 lines of frontmatter, one pass
- Large N case: reads at most 200 file frontmatters, still only 200 (not stat all files then pick 200)
- `readFileInRange` internally returns both content and mtime, so stat is integrated into the read path

ensureMemoryDirExists "pre-commitment" design (`memdir.ts`):

```
// memdir.ts
/**
 * Shared guidance text appended to each memory directory prompt line.
 * Shipped because Claude was burning turns on `ls`/`mkdir -p` before writing.
 * Harness guarantees the directory exists via ensureMemoryDirExists().
 */
export const DIR_EXISTS_GUIDANCE =
  'This directory already exists — write to it directly with the Write tool ' +
  '(do not run mkdir or check for its existence).'
```

Design intent: Without pre-creating directories, the model would spend 1-2 turns checking/creating directories before each memory write (burning turns). By calling `ensureMemoryDirExists` (guaranteed once at `loadMemoryPrompt` entry) to create directories upfront, and telling the model directly in the prompt that "the directory already exists, no need for mkdir," the saved turns can be used for actual work.

1.7 Why Plain Text over Database (Speculation)

1. Naturally compatible with Git workflow, achieving "code and cognition" version alignment
Plain text files naturally support Git diff, version history tracking, and PR review. If using a database, Git can only store binary diffs and cannot perform meaningful code review or change comparison.

2. Human readability and intervenability
With plain text format, developers can directly open files in an editor and read, correct, or supplement context just like modifying regular code.

3. Fully aligned with Agent's cognitive model and native capabilities
Plain text (like Markdown) is the format large language models are best at processing. Give it a path, and it can directly read, write, and edit. If a database is introduced, the agent also needs to master additional capabilities like database connections, table creation, schema migration, SQL queries, etc., adding unnecessary complexity.

4. No extra infrastructure or deployment cost, lowering cross-environment sync barrier
Plain text files depend on no external services — the simplest, lightest persistence solution. Moreover, in team collaboration, **plain text memory files can be directly pulled to new members' local environments via the code repository, achieving true "out-of-the-box" experience.**

5. Benefiting from the proliferation of ultra-long context windows in underlying models

2. File Structure Overview

2.1 Module Directory Structure

```
src/
├── memdir/                                # ★ Auto Memory Core Module
│   ├── memoryTypes.ts                     # Memory type enum, frontmatter parsing
│   ├── memoryScan.ts                      # Memory file scanning, manifest formatting
│   ├── memoryAge.ts                       # Memory freshness calculation
│   ├── memdir.ts                          # Memory prompt construction, MEMORY.md
truncation
│   ├── paths.ts                           # Memory path resolution, auto memory toggle
│   ├── teamMemPaths.ts                    # Team memory path security validation
│   ├── teamMemPrompts.ts                 # Team memory prompt composition
│   └── findRelevantMemories.ts           # AI relevance retrieval
```

```
│
├── services/
│   ├── SessionMemory/                      # ★ Session Memory Module
│   │   ├── sessionMemory.ts               # Extraction trigger logic, REPL hook
│   │   ├── sessionMemoryUtils.ts          # Global state, config, wait functions
│   │   └── prompts.ts                     # Session memory template, update prompt
│   ├── teamMemorySync/                     # ★ Team Memory Cloud Sync Module
│   │   ├── index.ts                       # pull/push/sync main logic
│   │   ├── watcher.ts                     # fs.watch debounced push
│   │   ├── secretScanner.ts               # Pre-upload sensitive info scanning
│   │   └── types.ts                       # Zod schema, API response types
│   ├── compact/
│   │   └── sessionMemoryCompact.ts        # Context compression based on session memory
│   ├── autoDream/                          # ★ Auto-Dream Background Integration Module
│   │   ├── autoDream.ts                   # Three-gate trigger + forked agent execution
│   │   ├── config.ts                      # isAutoDreamEnabled() toggle
│   │   ├── consolidationLock.ts          # Lock file (mtime = lastConsolidatedAt)
│   │   └── consolidationPrompt.ts        # Four-phase consolidation prompt
│   └── extractMemories/                    # ★ End-of-Session Memory Extraction Module
│       ├── extractMemories.ts              # Forked agent extraction main logic
│       └── prompts.ts                      # Extraction prompt template
│
├── tasks/DreamTask/
│   └── DreamTask.ts                        # DreamTask UI integration (footer pill)
│
├── tools/AgentTool/
│   ├── agentMemory.ts                      # Agent-level persistent memory
│   └── agentMemorySnapshot.ts              # Snapshot initialization and sync
│
├── components/memory/
│   ├── MemoryFileSelector.tsx              # Memory file selector UI component
│   └── MemoryUpdateNotification.tsx       # Memory update notification component
│
├── commands/memory/
│   └── memory.tsx                          # /memory CLI command implementation
│
└── utils/
    ├── claudemd.ts                         # CLAUDE.md loading main logic, @include
parsing
    └── memoryFileDetection.ts              # Memory file type auto-detection
```

---

3. Memory Type System Explained

3.1 Four Type Definitions

Defined in `src/memdir/memoryTypes.ts`, using **const assert closed-set enum**:

```
export const MEMORY_TYPES = [
  'user',       // User preferences, role, expertise
  'feedback',   // Work guidance (behaviors to avoid/repeat)
  'project',    // In-progress work, goals, initiatives
  'reference'   // External system references/pointers
```

```
] as const

export type MemoryType = (typeof MEMORY_TYPES)[number]
// Inferred as: 'user' | 'feedback' | 'project' | 'reference'
```

3.2 Scope and Save Specifications per Type

Type	Default Scope	When to Promote to Team	Example Content
`user`	Private (always private)	Never	User is a senior Go engineer, first time touching React
`feedback`	Private (default)	Only when team consensus needed	Integration tests must hit real DB (mock caused production incident)
`project`	Team (strong bias)	Usually team-scoped	Linear project "INGEST" tracks all pipeline bugs
`reference`	Team (usually)	Usually team-scoped	Grafana dashboard grafana.internal/d/api-latency is oncall monitoring

PS: Private memories (user / feedback / project / reference) exist only on local disk; only **Team Memory** (shared portion) syncs to the cloud for sharing among org members of the same repo.

3.3 Frontmatter Format Specification

Each memory is a Markdown file and must include YAML frontmatter:

PS: Uses standard YAML frontmatter (--- delimiter), matched directly via regex.

```
---
name: {{unique memory name — use snake_case or kebab-case}}
description: {{one-line description — the more specific, the better for AI relevance
judgment}}
type: {{user | feedback | project | reference}}
---

Memory body content.

For feedback/project type, recommended format:
Rule/fact itself
**Why:** Reason — usually a past event or strong preference
**How to apply:** When and where to apply this rule
```

> **YAML special character auto-quoting design** (`frontmatterParser.ts`):
>
> Source comment explains the rationale:
>
> ```
> const YAML_SPECIAL_CHARS = /[{}[]*&#!|>%@`]|: /
> // - { } are flow mapping indicators
> // - * is anchor/alias indicator
> // - [ ] are flow sequence indicators
> // - ': ' (colon followed by space) is key indicator
> //   causes 'Nested mappings are not allowed in compact mappings'
> // - # is comment indicator
> ```

> Typical problem scenario: In `paths: "**\\/*.{ts,tsx}"`, the `*`, `{`, `}`, `/` are all YAML special characters; without handling, writing them directly into frontmatter causes YAML parser errors.
>
> `quoteProblematicValues()` automatically retries on parse failure, wrapping all values containing special characters in double quotes. Values are pre-quoted on write, ensuring round-trip consistency.

3.4 Memory Drift Warning

When memory content conflicts with current code state, the system requires: (current situation takes priority over memory when it comes to timeliness)

> Trust observed reality (what you observe now), not records in memory — update or delete outdated memories; do not make assertions based on outdated information.

3.5 Parsing Functions

```
// memoryTypes.ts
export function parseMemoryType(raw: unknown): MemoryType | undefined {
  // Invalid or missing value returns undefined, no exception thrown
  if (typeof raw !== 'string') return undefined
  // Second line: find exact match in ['user', 'feedback', 'project', 'reference']
  return MEMORY_TYPES.find(t => t === raw)
}
```

---

4. Storage Mechanism: File System Architecture

4.1 Directory Structure (Three-Layer Scope)

```
~/.claude/                                  # User-level (global private)
├── memory/
│   ├── MEMORY.md                           # Index manifest (entrypoint)
│   ├── user_role.md                        # Memory files *.md
│   ├── feedback_testing.md
│   ├── project_ingest.md
│   ├── reference_linear.md
│   └── team/                               # Team-shared memory
│       ├── MEMORY.md
│       └── *.md
│
├── agent-memory/                           # Agent persistent memory (user scope)
│   └── <agentType>/
│       └── MEMORY.md
│
└── agent-memory-local/                     # Agent persistent memory (local scope)
    └── <agentType>/
        └── MEMORY.md

<git-root>/.claude/                        # Project-level (shared within project)
├── memory/
│   ├── MEMORY.md
│   ├── *.md
│   └── team/
```

```
│        ├── MEMORY.md
│        └── *.md
│
└── agent-memory/                          # Agent persistent memory (project scope)
    └── <agentType>/
        └── MEMORY.md
```

Differentiation:

```
~/.claude/projects/<project>/memory/ — 用户 Auto Memory

- 作用域：用户 × 项目（跨会话）
- 路径：~/.claude/projects/<sanitized-git-root>/memory/
- 性质：用户个人的记忆，存在于用户配置目录下
- 可见性：只有该用户自己能看到
- 团队共享：不共享（不在项目目录内）
- 记忆范围：user / feedback / project / reference 四类，这是 prefetch 和 findRelevantMemories 操作的主要目录

~/.claude/agent-memory/<agentType>/ — Agent Memory (user scope)

- 作用域：agent × 用户全局（跨所有项目）
- 路径：~/.claude/agent-memory/<agentType>/
- 性质：agent 的持久化记忆，跟 agent 类型走，不跟项目走
- 可见性：只有该 agent 自己能看到
- 团队共享：不共享

<cwd>/.claude/agent-memory/<agentType>/ — Agent Memory (project scope)

- 作用域：agent × 当前项目（跨会话）
- 路径：在项目根目录的 .claude/agent-memory/<agentType>/
- 性质：agent 记忆，跟项目走，会被提交到 git
- 可见性：团队所有成员共享同一 agent 实例的上下文时可见

<cwd>/.claude/agent-memory-local/<agentType>/ — Agent Memory (local scope)

- 作用域：agent × 当前机器 × 当前项目
- 路径：在项目目录，gitignored（.claude 通常在 gitignore 里）
- 性质：本地专用，不同步到 VCS
- 可见性：只有当前机器的当前项目能访问
```

4.2 MEMORY.md Index File (Entrypoint)

Nature: **MEMORY.md is an index manifest, not a memory container**. Unlike other systems (e.g., Obsidian) — content should NOT be written directly into MEMORY.md.

Format specification:

```
- [Title](filename.md) — one-line description (optional)
- [user] user_role.md (2026-03-30T10:00:00.000Z): User is a data scientist
- [feedback] feedback_testing.md (2026-03-29T08:00:00.000Z)
```

Hard limits (`memdir.ts`):

- Max 200 lines
- Max 25,000 bytes

When limits are exceeded, `truncateEntrypointContent()` applies both line-count and byte-count truncation simultaneously, recording `wasLineTruncated` and `wasByteTruncated` flags.

Truncation appends a warning:

```
> WARNING: MEMORY.md is 347 lines and 94KB (limit: 200 lines and 25KB).
    Only part of it was loaded. Keep index entries to one line under ~200 chars;
    move detail into topic files.
```

PS: Information is not completely lost — only the index entry in MEMORY.md is lost. So even if an entry is truncated, if the user asks about related content, the Sonnet relevance selector can still read that topic file's frontmatter from disk and retrieve it.

4.3 Auto Memory Directory Path Resolution

Full path resolution (`getAutoMemPath()` , priority high to low):

```
  1. CLAUDE_COWORK_MEMORY_PATH_OVERRIDE (env var) → return full path directly, no
concatenation
  2. autoMemoryDirectory config option (settings.json, priority: policySettings >
flagSettings > localSettings > userSettings)
    - ⚠  Project-level .claude/settings.json is PROHIBITED from setting this field
(security: prevents malicious repos from redirecting memory path to ~/.ssh or other
sensitive directories)
  3. Default path:
  <memoryBase>/projects/<sanitized-git-root>/memory/
    - sanitizePath(): converts git root path to safe directory name (non-alphanumeric chars
replaced with -, truncates long names and appends hash)
    - getAutoMemBase(): uses git root instead of CWD, ensuring all worktrees of the same
repo share the same memory directory

  Collaboration sharing:
  - All workspaces under the same git root share the same auto memory
  - Team memory additionally uses <autoMemPath>/team/ directory (via TEAMMEM feature branch)
```

4.4 Team Memory Path

`src/memdir/teamMemPaths.ts` — `getTeamMemPath()` :

```
<memoryBase>/projects/<sanitized-project-root>/memory/team/
```

PS:

This team memory stores server-side status information and the like, i.e., public information accessible to anyone with permission

```
Team Memory Sharing Mechanism

1. Core Definition and Granularity
Team Memory enables memory sharing among all collaborators of the same Git repository.
Identification granularity: git remote URL (owner/repo) as unique identifier.
Not explicitly defined: does not rely on traditional "team" settings, but on repository
access permissions.

2. Access and Authentication
Admission: Any member with access to the repository.
Auth: OAuth authentication to obtain tokens, enabling read/write of shared memories.

3. Sync Mechanism (Pull-Push Mode)
The system ensures data consistency across devices via the following flow:
Session start: Pull operation, fetch latest version from server and overwrite local data.
Session end: Push operation, sync local incremental changes back to server.

4. Conflict Handling and Security
Conflict management: ETag optimistic locking effectively resolves data conflicts from
concurrent edits.
```

```
Privacy protection: Pre-upload file scanning. If API keys, tokens, or other sensitive
credentials are detected, the file is automatically skipped, preventing data leakage.

5. Technical Characteristics
Independence: Does not occupy Git version history, no dependency on local filesystem
sharing.
Cloud-driven: Purely relies on API and OAuth Token for cross-device sync.
```

---

5. Core Data Structures

5.1 MemoryHeader (Scan Result)

```
// src/memdir/memoryScan.ts
export type MemoryHeader = {
  filename: string           // filename (no path)
  filePath: string            // absolute path
  mtimeMs: number            // file modification time (Unix epoch in ms)
  description: string | null // description from frontmatter (for manifest)
  type: MemoryType | undefined // type from frontmatter (may be missing)
}
```

Purpose: Return type of `scanMemoryFiles()`, used for manifest formatting and **relevance retrieval**. **Only reads the first 30 lines of frontmatter, not full content**.

5.2 RelevantMemory (Retrieval Result)

```
// src/memdir/findRelevantMemories.ts
export type RelevantMemory = {
  path: string    // absolute path to memory file
  mtimeMs: number // modification time (for freshness calculation)
}
```

Retrieval process detailed in [6. Retrieval and Search Algorithm]

5.3 MemoryFileInfo (Complete Memory File)

```
// src/utils/claudemd.ts
export type MemoryFileInfo = {
  path: string                        // absolute file path
  type: MemoryType                    // 'Managed' | 'User' | 'Project' | 'Local'
                                       // 'AutoMem' | 'TeamMem'
  content: string                     // parsed plain text content
  parent?: string                      // parent file path that included this file (for
@include tracking)
  globs?: string[]                    // paths field in frontmatter (conditional rules)
  contentDiffersFromDisk?: boolean    // whether in-memory content differs from disk
  rawContent?: string                 // raw Markdown content (with frontmatter)
}
```

5.4 SyncState (Team Memory Sync State)

```
// src/services/teamMemorySync/index.ts
export type SyncState = {
  lastKnownChecksum: string | null     // ETag, for HTTP conditional requests (If-None-Match); used for next pull's If-None-Match (whole-repo level: has anything changed?)

  serverChecksums: Map<string, string> // key (relative path) -> SHA-256 hash; used for this push's delta calculation (file level: which files changed?)

  serverMaxEntries: number | null      // server's max entry limit (learned from 413 responses); used for pre-push local truncation (avoid triggering 413)
}
```

PS: See [9. Team Memory Cloud Sync System](#) for details.

- To sync the latest team memory on the server
- Team memory is stored on the server, essentially a shared memory database. Each member's local is just a cache replica — pull syncs latest, push merges local changes up.

```
The server computes a SHA-256 digest of the entire team memory snapshot and returns it each time push succeeds.

Purpose is conditional requests:

Local pull:
  Request header carries If-None-Match: "<previous checksum>"
  Server compares:
    checksum unchanged → return 304 Not Modified (no change, download nothing)
    checksum changed → return full data + new checksum

This way each pull doesn't need to download full content; data is only transferred when something actually changed.
```

5.5 SyncResult Response Types

```
// src/services/teamMemorySync/types.ts
export type TeamMemorySyncFetchResult = {
  success: boolean
  data?: TeamMemoryData
  isEmpty?: boolean
  notModified?: boolean        // HTTP 304, equivalent to last response
  checksum?: string            // ETag
  error?: string
  skipRetry?: boolean          // permanent failure, skip retry
  errorType?: 'auth' | 'timeout' | 'network' | 'parse' | 'unknown'
  httpStatus?: number
}

export type TeamMemorySyncPushResult = {
  success: boolean
  filesUploaded: number
  checksum?: string
  conflict?: boolean          // HTTP 412, need to refresh checksum and retry
```

```
  error?: string
  skippedSecrets?: SkippedSecretFile[] // skipped files containing sensitive info
  errorType?: 'auth' | 'timeout' | 'network' | 'conflict' |
              'unknown' | 'no_oauth' | 'no_repo'
  httpStatus?: number
}
```

PS:

- Team shared knowledge base file sync service — responsible for uploading local files to server (push) and fetching from server (pull)
- **Sensitive info is sanitized before uploading memories**, see 9.6 Secret Scanner for details.

5.6 SessionMemoryConfig (Session Memory Configuration)

```
// src/services/SessionMemory/sessionMemoryUtils.ts
export type SessionMemoryConfig = {
  minimumMessageTokensToInit: number      // minimum token count to start initialization
  minimumTokensBetweenUpdate: number     // minimum token increment between updates
  toolCallsBetweenUpdates: number        // minimum tool call count between updates
}

export const DEFAULT_SESSION_MEMORY_CONFIG: SessionMemoryConfig = {
  minimumMessageTokensToInit: 10000,
  minimumTokensBetweenUpdate: 5000,
  toolCallsBetweenUpdates: 3,
}
```

---

6. Retrieval and Search Algorithm

6.1 Memory File Scanning (scanMemoryFiles)

File: `src/memdir/memoryScan.ts`

```
export async function scanMemoryFiles(
  memoryDir: string,
  signal: AbortSignal,
): Promise<MemoryHeader[]>
```

Algorithm steps:

1. Call `readdir(memoryDir, { recursive: true })` to traverse directory
2. Filter all `.md` files (exclude MEMORY.md itself)
3. For each file:
   - Call `readFileInRange(filePath, { maxLines: 30 })` to read only first 30 lines -> corresponding to frontmatter
   - Call `parseFrontmatter(raw)` to parse frontmatter
   - `readFileInRange` internally returns `mtimeMs` at the same time (no separate stat needed)
4. Extract `{ filename, filePath, mtimeMs, description, type }`
5. Sort by `mtimeMs` descending (newest first)

6. Truncate to max 200 files (`MAX_MEMORY_FILES`)
7. Return `MemoryHeader[]`

> **Single-pass read-then-sort optimization**: Source comment (`memoryScan.ts`) explains this design:
>
> ```
> // Single-pass: readFileInRange stats internally and returns mtimeMs, so we
> // read-then-sort rather than stat-sort-read. For the common case (N ≤ 200)
> // this halves syscalls vs a separate stat round; for large N we read a few
> // extra small files but still avoid the double-stat on the surviving 200.
> ```

Manifest formatting:

```
export function formatMemoryManifest(memories: MemoryHeader[]): string {
  return memories.map(m => {
    const tag = m.type ? `[${m.type}] ` : ''
    const ts = new Date(m.mtimeMs).toISOString()
    return m.description
      ? `- ${tag}${m.filename} (${ts}): ${m.description}`
      : `- ${tag}${m.filename} (${ts})`
  }).join('\n')
}
```

Output example:

```
- [user] user_role.md (2026-03-30T10:00:00.000Z): User is a data scientist focused on
observability
- [feedback] feedback_testing.md (2026-03-29T08:00:00.000Z)
- [project] project_ingest.md (2026-03-28T15:30:00.000Z): Linear project INGEST tracks
pipeline bugs
```

6.2 AI Relevance Retrieval (findRelevantMemories)

File: `src/memdir/findRelevantMemories.ts`

This is the core intelligence layer of the memory system, using the Sonnet model for side queries.

```
export async function findRelevantMemories(
  query: string,                    // current user message/question
  memoryDir: string,                 // memory directory
  signal: AbortSignal,
  recentTools: readonly string[] = [],  // recently used tools (context signal)
  alreadySurfaced: ReadonlySet<string> = new Set(), // already shown memories (dedup)
): Promise<RelevantMemory[]>
```

Algorithm steps:

```
Step 1: Call scanMemoryFiles() to get manifest of all memories
Step 2: Format manifest (with timestamps and descriptions)
Step 3: Build System Prompt (SELECT_MEMORIES_SYSTEM_PROMPT):
        "You are a memory retrieval assistant. Given a user query and a list of
         available memories, select up to 5 memories most relevant to the query.
         Prefer memories with recent timestamps and direct topic matches.
```

```
         Output JSON: { "selected_memories": ["filename1", "filename2", ...] }
         Exclude: memories already surfaced in this conversation."

Step 4: Send request using Sonnet model (sideQuery)
Step 5: Parse JSON response, extract selected_memories array
Step 6: Filter: exclude memories already shown in alreadySurfaced
Step 7: Return RelevantMemory[] (path + mtimeMs)
```

Key design: Side Query means relevance retrieval happens in an independent call, consuming no main conversation tokens and not affecting the main reasoning flow.

6.3 Conditional Rules

`claudemd.ts` supports the `paths` field in frontmatter for conditional memories:

```
---
name: vscode-settings-hint
description: VSCode special settings hint
type: user
paths:
  - "**/.vscode/**"
  - "**/settings.json"
---

When the user edits VSCode configuration...
```

The system matches the current working directory path against `glob` patterns, injecting the memory only when matched.

PS:
Typical **context-aware** or **dynamic prompt injection** design. Simply put:

- Normal memories are "globally active" — once loaded, always present in context
- Conditional rules allow memory files to declare: **"this memory is only useful when I'm editing files at specific paths, so only inject when matched"**
  Personal speculation: this is a powerful feature. Bound to paths, it can achieve the following:
- Different projects, file types, work phases — automatically switch different behavioral norms and prompts without manual user specification
- Teams can declare project constitutions in team memory; each member entering the workspace, Claude Code automatically operates based on that constitution
- Using lazy-loading approach, this feature becomes an activation/loading trigger for memories. If the path is a skill, store that skill's memory; if it's an MCP tool, store that MCP tool's memory; if it's a project file, it becomes that project's local memory. This strategy can store diverse memories without causing context issues from too many memory index entries

---

7. Memory Staleness Mechanism

File: `src/memdir/memoryAge.ts`

7.1 Algorithm Implementation

```
// milliseconds per day: 86_400_000
export function memoryAgeDays(mtimeMs: number): number {
  return Math.max(0, Math.floor((Date.now() - mtimeMs) / 86_400_000))
}
// Formula: (current time - file modification time) / milliseconds per day, floor gives days
// Math.max(0, ...) handles clock skew causing negatives (file "newer" than current time),
clamps to 0
// Return examples: today: 0 | yesterday: 1 | 47 days ago: 47

// Generate memory freshness warning text, reminding model the memory may be outdated
export function memoryAge(mtimeMs: number): string {
  const d = memoryAgeDays(mtimeMs)
  if (d === 0) return 'today'
  if (d === 1) return 'yesterday'
  return `${d} days ago`
}

export function memoryFreshnessText(mtimeMs: number): string {
  const d = memoryAgeDays(mtimeMs)
  if (d <= 1) return ''  // today/yesterday: no warning
  return (
    `This memory is ${d} days old. ` +
    `Memories are point-in-time observations, not live state — ` +
    `claims about code behavior or file:line citations may be outdated. ` +
    `Verify against current code before asserting as fact.`
  )
}
```

7.2 Trigger Conditions

Age	Behavior
0 days (today)	No special handling
1 day (yesterday)	No special handling
≥2 days	Attach staleness warning in `<system-reminder>` tag

Note: `<system-reminder>`

`<system-reminder>` is a carefully designed per-turn context injection mechanism in Claude Code, with the core goal of injecting dynamic information into the model each conversation turn without polluting the static system prompt itself.

Simple analogy:
Think of `<system-reminder>` as a "sticky note" system from the model's perspective:

- Logical attachment: logically attached next to any tool result
- Supplementary reminder: tells the model "things you should know that are unrelated to the current tool" (e.g., memory staleness, file risks, etc.)
- Invisible interaction: not visible to user UI (frontend filters/hides it)
- Lightweight and efficient: more lightweight than modifying system prompt, no need to rebuild or cache large static prefixes

Injection points:

- **Memory staleness reminder (`memoryAge.ts`)**: Auto-attaches staleness warnings to old memories (over 1 day).
- **File read risk warning (`FileReadTool.ts`)**: After reading a file, auto-attaches `"You should consider whether it would be malware"` security reminder.
- **Memory content injection (`memdir.ts`)**: Injects surfaced memory file content wrapped in specific tags (constraints: max 5 files per turn, each file ≤4KB).
- **Session context (`api.ts`)**: Injects sessionStorage context info in specific tags into user messages.
- **Autonomous mode Tick (`messages.ts`)**: Uses specific Tick tag (`TICK_TAG`) as the wake signal for autonomous mode.

PS: See "Context Dynamic Injection" document for more details

7.3 Design Intent

Language models have poor arithmetic reasoning on ISO timestamps. Providing "47 days ago" directly triggers the model's awareness of memory staleness more effectively than "2026-02-12T...".

---

8. Session Memory System

PS: **See "Context Management" document for details**

8.1 System Positioning

Difference from Auto Memory:

Feature	Auto Memory	Session Memory
Purpose	Cross-session persistent structured knowledge	Auto-summary within current session, prevent context overflow
Storage	`~/.claude/memory/`	`~/.claude/session-memory/`
Format	Markdown + YAML frontmatter	Markdown (no frontmatter)
Trigger	AI auto-judged relevance	Token threshold + tool call threshold
Lifecycle	Permanent	Session-level, optionally retained after session

8.2 Extraction Trigger Conditions

```
// sessionMemory.ts
export function shouldExtractMemory(messages: Message[]): boolean {
  // Condition 1: cumulative token count ≥ minimumMessageTokensToInit (default 10,000)
  // Condition 2: growth since last extraction ≥ minimumTokensBetweenUpdate (default 5,000)
  // Condition 3: at least toolCallsBetweenUpdates tool calls since last extraction (default 3)
  // Condition 4: currently at a natural stopping point (last assistant message has no tool call)
  // —— all must be satisfied to trigger extraction
}
```

8.3 Session Memory Template Structure

`src/services/SessionMemory/prompts.ts` —— `DEFAULT_SESSION_MEMORY_TEMPLATE`：

```
# Session Title
## Current State
## Task specification
## Files and Functions
## Workflow
## Errors & Corrections
## Codebase and System Documentation
## Learnings
## Key results
## Worklog
```

The system dynamically calculates each section's token size. If total length exceeds `MAX_TOTAL_SESSION_MEMORY_TOKENS` (12,000), it trims by priority (worklog > code docs > intermediate steps).

8.4 Extraction Execution Flow

```
// sessionMemoryUtils.ts
// Global state (session-level)
let lastSummarizedMessageId: string | undefined
let extractionStartedAt: number | undefined
let tokensAtLastExtraction = 0
let sessionMemoryInitialized = false

// Main extraction function (sequential, prevents concurrency)
const extractSessionMemory = sequential(async function (context) {
  // 1. Get current session messages
  // 2. Check shouldExtractMemory() —— four conditions
  // 3. Call AI model, passing:
  //    - current session-memory file content (if any)
  //    - update prompt (incremental update, not full rewrite)
  //    - template structure
  // 4. Write AI output to ~/.claude/session-memory/<sessionId>.md
  // 5. Update global state
})
```

8.5 Session Memory Compaction

File: `src/services/compact/sessionMemoryCompact.ts`

When context is about to overflow, the system uses summary info from session memory to decide which messages to retain:

```
// Priority (high to low):
// 1. File change records (git diff output)
// 2. Error messages
// 3. User explicit constraints/preferences
// 4. Decision records
// —— Lowest priority: intermediate reasoning steps, repetitive operation patterns

export function calculateMessagesToKeepIndex(
  messages: Message[],
  lastSummarizedIndex: number,
): number {
```

```
  // Returns the starting index of messages to keep
  // Auto-adjusts: ensures not cutting tool_use / tool_result pairs
}
```

Compaction trigger threshold (configurable):

```
export const DEFAULT_SM_COMPACT_CONFIG = {
  minTokens: 10_000,           // minimum message token count
  minTextBlockMessages: 5,     // minimum text message count
  maxTokens: 40_000,           // max tokens after compaction
}
```

---

9. Team Memory Cloud Sync System

9.1 API Endpoints

Method	Endpoint	Purpose
GET	`/api/claude_code/team_memory?repo={owner/repo}`	Fetch team memory
GET	`/api/claude_code/team_memory?repo={owner/repo}&view=hashes`	Fetch hash metadata only (for delta calculation)
PUT	`/api/claude_code/team_memory?repo={owner/repo}`	Batch upload/update memories

9.2 Sync State Machine

```
export type SyncState = {
  lastKnownChecksum: string | null   // ETag (for If-None-Match conditional request). If server content unchanged, returns 304, skips writing to local (saves I/O)
  serverChecksums: Map<string, string> // key -> SHA-256(hex). Before each local push, compare this Map — only upload files whose hash changed, skip others
  serverMaxEntries: number | null     // Server's entry limit. Local remembers this number to avoid triggering 413 again next push
}
```

9.3 Full Sync Algorithm

`syncTeamMemory()` = `pullTeamMemory()` + incremental `pushTeamMemory()`

Pull flow:

```
1. Check if syncState.serverChecksums is empty
   → empty: full pull (first-time sync)
   → non-empty: conditional request with If-None-Match: <checksum>
2. HTTP 200: write to local files, return filesWritten
3. HTTP 304 (notModified): no action needed
4. HTTP 413 (too_many_entries): extract max_entries from response, update serverMaxEntries
5. Other errors: return error + errorType
```

Push flow (incremental delta):

```
1. Read all files in local team memory directory
2. Calculate SHA-256(content) for each file
```

```
3. Compare with serverChecksums Map (key -> hash):
   → same hash: skip
   → different hash or new file: add to delta
4. secretScanner.scanForSecrets() —— scan all delta files
   → sensitive info found: skip file, add to skippedSecrets[]
5. batchDeltaByBytes() —— batch by MAX_PUT_BODY_BYTES (200KB)

> **Why 200KB limit?** Source comment (`teamMemorySync/index.ts`) reveals the history:
> ```
> // #21969 removed the client entry-count cap; cold pushes from heavy users
> // then sent 300KB-1.4MB bodies and hit the gateway body-size cap.
> // 200KB leaves headroom under the observed threshold and keeps a single
> // -entry-at-MAX_FILE_SIZE_BYTES solo batch (~250KB) just under the real
> // gateway limit.
> ```
>
> Background: There used to be a client entry-count hard cap, but it was removed (#21969).
After that, heavy users started sending 300KB-1.4MB bodies, triggering the API Gateway's
body-size limit (~256-512KB, timeout ~750ms vs normal 2.3s).
>
> Solution: Set 200KB soft cap, batch on client side so no single request exceeds the
gateway limit. `MAX_FILE_SIZE_BYTES = 250KB` is the single-file limit, so a single-file solo
batch (~250KB) would just exceed 200KB but might pass (headroom left).

6. Send PUT request (with If-Match: <lastKnownChecksum>)
7. HTTP 200: update syncState (checksum + serverChecksums)
8. HTTP 412 (conflict): re-request view=hashes, refresh serverChecksums, recalculate delta,
retry (up to 2 times)
```

9.4 Hash Algorithm

```
// src/services/teamMemorySync/index.ts
export function hashContent(content: string): string {
  // Uses SHA-256, returns hexadecimal string
  // Used for:
  //   - comparing local files with server files
  //   - calculating ETag (lastKnownChecksum)
}
```

9.5 File System Watcher

`src/services/teamMemorySync/watcher.ts`：

```
Startup timing: on first write to team memory file
Scope: team memory directory (recursive)
Debounce delay: DEBOUNCE_MS = 2000ms

Flow:
1. fs.watch({ recursive: true }, callback) listens for create/modify/change events
2. Any event triggers: reset debounceTimer (setTimeout 2000ms)
3. No new events within 2000ms: executePush()
4. If pushInProgress: set hasPendingChanges = true, wait for current push to complete before
pushing
```

```
5. Permanent failure (no_oauth/no_repo/4xx non-409/429): set pushSuppressedReason, no
retries for subsequent writes
```

9.6 Secret Scanner

`src/services/teamMemorySync/secretScanner.ts`:

Executes pre-upload, ensuring sensitive information never leaves the local machine.

```
export function scanForSecrets(content: string): SecretMatch[]
// Returns: { ruleId: string, label: string }[]
// Does not log specific sensitive values, only records rule ID and label

// Scanning rules (gitleaks subset, ~30+ rules):
// AWS_ACCESS_KEY, AWS_SECRET_KEY, GCP_API_KEY, AZURE_TOKEN,
// DIGITALOCEAN_TOKEN, ANTHROPIC_API_KEY, OPENAI_API_KEY,
// HUGGINGFACE_TOKEN, GITHUB_PAT, GITLAB_TOKEN,
// SLACK_TOKEN, TWILIO_TOKEN, SENDGRID_KEY,
// NPM_TOKEN, PYPI_TOKEN, DATABRICKS_TOKEN,
// GRAFANA_TOKEN, SENTRY_DSN, STRIPE_KEY,
// SHOPIFY_SECRET, SSH_PRIVATE_KEY, PGP_PRIVATE_KEY,
// MONGODB_URI, POSTGRESQL_STRING, ...
```

Rules use precompiled regex (`compiledRules`), avoiding recompilation on each scan.

PS: No LLM call — pure code regex matching.

---

10. Agent Memory System

PS: Agent Memory is **memory between a specific named Agent and its tasks** (temporarily created sub-agents don't need or get long-term memory configured).
Built-in named agents (persistent agent instances created via /agent create):

- Explore — specialized for code exploration, read-only, no file modification
- Plan — makes plans, plans before executing
- General Purpose — default all-purpose agent
- Verification — verification agent (runs tests, etc.)
- Statusline Setup — status bar configuration
- Code Guide — embedded Claude Code usage guide

10.1 Three Scopes

```
// src/tools/AgentTool/agentMemory.ts
export type AgentMemoryScope = 'user' | 'project' | 'local'
```

Scope	Path	Purpose	Core Design Intent
`user`	`~/.claude/agent-memory/<agentType>/`	Global private, shared across projects	Knowledge I apply in all projects
`project`	`<cwd>/.claude/agent-memory/<agentType>/`	Shared within project, committed to git, shared with team	Knowledge shared between me and the team on this project

Scope	Path	Purpose	Core Design Intent
`local`	`<cwd>/.claude/agent-memory-local/<agentType>/`	Valid only locally when no mount; with mount, syncs local memory to remote mount point via fixed path, shared with team	Knowledge belonging only to this machine/me (machine-specific stuff)

10.2 Snapshot System

File: `src/tools/AgentTool/agentMemorySnapshot.ts`

The snapshot mechanism syncs memory updates from external sources (e.g., remote config system) without disrupting existing memories:

```
// Snapshot directory structure
<cwd>/.claude/agent-memory-snapshots/<agentType>/
└── snapshot.json           # Snapshot metadata (contains updatedAt)

// Sync state tracking
<cwd>/.claude/agent-memory/<agentType>/
└── .snapshot-synced.json   # Records last synced snapshot timestamp

export async function checkAgentMemorySnapshot(
  agentType: string,
  scope: AgentMemoryScope,
): Promise<{
  action: 'none' | 'initialize' | 'prompt-update'
  snapshotTimestamp?: string
}>
```

Decision logic:

- `none` : no snapshot or already synced, no action
- `initialize` : local directory is empty, initialize from snapshot
- `prompt-update` : snapshot is newer than synced version, prompt user to confirm update

---

11. @include Directive Explained

PS:

- Memory content can @include another file; when reading memory, the full content is expanded
- More intuitively: @include is the import statement in memory files, allowing one memory to reference another, supports nesting with loop and depth protection

11.1 Syntax

```
@path                    # resolved from memory base path (~/.claude)
@./relative/path         # relative to current file's directory
@~/home/absolute/path    # expands ~ to user home directory
@/absolute/path          # absolute path
```

11.2 Parsing Flow (claudemd.ts)

```
// MAX_INCLUDE_DEPTH = 5 (prevents circular reference explosion)

export async function processMemoryFile(
  filePath: string,
  type: MemoryType,
  processedPaths: Set<string>,  // track processed paths, prevent loops
  includeExternal: boolean,
  depth: number = 0,           // recursion depth counter
  parent?: string,             // parent file path (for parent field)
): Promise<MemoryFileInfo[]> {
  if (depth > MAX_INCLUDE_DEPTH) return []
  if (processedPaths.has(filePath)) return []  // loop detection
  processedPaths.add(filePath)

  // 1. Read file
  // 2. Parse frontmatter
  // 3. Extract paths from @include directives
  // 4. Recursively process each included file
  // 5. Return all MemoryFileInfo (with parent field)
}
```

11.3 Supported File Types

The system maintains a `TEXT_FILE_EXTENSIONS` set (100+ extensions), including: `.md`, `.txt`, `.json`, `.yaml`, `.yml`, `.toml`, `.xml`, `.html`, `.css`, `.js`, `.ts`, `.py`, `.rb`, `.go`, `.rs`, `.java`, `.c`, `.cpp`, `.h`, `.sh`, `.bash`, `.zsh`, `.ps1`, `.sql`, `.graphql`, `.proto`, `.env`, `.ini`, `.cfg`, `.conf`, etc.

11.4 Conditional @include (paths field)

The `paths` field in frontmatter supports glob patterns:

```
---
name: docker-hints
description: Docker-related development hints
type: reference
paths:
  - "**/Dockerfile"
  - "**/docker-compose.yml"
  - "**/.dockerignore"
---

Inject this memory when the user handles Docker-related files...
```

The system checks if the current working directory path matches any pattern via `resolveExcludePatterns()`.

---

12. Memory Loading Order and Priority

`src/utils/claudemd.ts` — `getMemoryFiles()`:

12.1 Four-Level Loading Order

```
Priority 1 (lowest) → Managed (global directives)
  Path: /etc/claude-code/CLAUDE.md
  Description: System-level global directives, apply to all users

Priority 2 → User (user global directives)
  Path: ~/.claude/CLAUDE.md
  Description: User's private global directives

Priority 3 → Project (project directives)
  Paths:
    <cwd>/CLAUDE.md
    <cwd>/.claude/CLAUDE.md
    <cwd>/.claude/rules/*.md
  Description: Can be committed to version control, shared within project

Priority 4 (highest) → Local (local private)
  Paths:
    <cwd>/CLAUDE.local.md
    <cwd>/.claude/rules.local/*.md
  Description: Should NOT be committed to version control (usually in .gitignore)
```

Closer to CWD Wins principle:
For multiple files of the same type, later-loaded ones override earlier ones. Example: if both `CLAUDE.md` and `.claude/CLAUDE.md` exist in the project directory, the latter has higher priority.

12.2 Content Merge Strategy

The system uses **override merge** (not concatenation):

- For the same key (e.g., same section name), later content overrides earlier
- For different keys, content is merged and retained
- Final output is a single string

PS: Override merge solves the problem of **which content wins when multiple memory files conflict**, not the problem of preventing duplicate saving of the same memory.

---

13. Security Mechanisms

13.1 Path Traversal Protection

`src/memdir/teamMemPaths.ts` — `validateTeamMemWritePath()`:

```
async function validateTeamMemWritePath(filePath: string): Promise<string> {
  // 1. Check null bytes (\0): inserting \0 in path, many language low-level APIs treat it
as string terminator, causing actual path truncation
  // 2. Normalize path (resolve . and ..)
  // 3. realpathDeepestExisting() —— resolve to nearest existing ancestor directory
  //    - prevents traversing directory structure via symlinks
  // 4. Verify normalized path is still within team memory directory
  // 5. Reject Unicode normalization attacks (NFKC/NFKC_CaseFold): visually identical but
different character codes, bypasses string checks
  //    - ".." (U+2024) vs ".." (U+002E U+002E)
```

```
  // 6. Reject URL-encoded path traversal (%2e%2e/%2f): %2e%2e%2f decodes to ../; some validation logic checks the string without first decoding URL
  // }
```

PS:

- This is **path traversal attack protection**: when uploading files, verifies the path won't escape the team memory directory — blocks various attack vectors like .. traversal, symlink traversal, Unicode confusion, URL encoding bypass
- **Path traversal attack**: if an attacker can write a path as ../.ssh/authorized_keys, they could overwrite /home/user/.ssh/authorized_keys — writing files **outside** the team memory directory

13.2 Config Injection Protection

`src/memdir/paths.ts`:

- `autoMemoryDirectory` is **prohibited** from being set in project-level `settings.json`
- Can only be injected from the three safe levels: `policy`, `local`, `user`
- Reason: project settings can be controlled by arbitrary repository submitters, could point to `~/.ssh/` or other sensitive directories

13.3 Sensitive Information Protection

See 9.6 Secret Scanner: Client-side scanning ensures sensitive info never leaves local machine.

13.4 File Permissions

- Memory directory: `0o700` (owner read/write/execute only)
- Memory files: `0o600` (owner read/write only)

---

14. Complete Function Index

14.1 Auto Memory Module (memdir/)

File	Function Signature	Purpose
`memoryTypes.ts`	`parseMemoryType(raw: unknown)`	Parse frontmatter type value
`memoryTypes.ts`	`TYPES_SECTION_COMBINED` (constant)	Type description prompt with scope tags
`memoryTypes.ts`	`TYPES_SECTION_INDIVIDUAL` (constant)	Type description prompt without scope tags
`memoryScan.ts`	`scanMemoryFiles(memoryDir, signal)`	Recursive scan, return MemoryHeader[]
`memoryScan.ts`	`formatMemoryManifest(memories)`	Format as standard manifest text
`memoryAge.ts`	`memoryAgeDays(mtimeMs)`	Calculate age (days, floor)
`memoryAge.ts`	`memoryAge(mtimeMs)`	Human-readable age ("today"/"N days

File	Function Signature	Purpose
		ago")
`memoryAge.ts`	`memoryFreshnessText(mtimeMs)`	Staleness warning text (triggered at ≥2 days)
`memoryAge.ts`	`memoryFreshnessNote(mtimeMs)`	Staleness warning (with `<system-reminder>` tag)
`memdir.ts`	`truncateEntrypointContent(raw)`	Dual-limit truncation (lines + bytes)
`memdir.ts`	`buildMemoryLines(displayName, memoryDir, extraGuidelines?, skipIndex?)`	Build type memory description (without MEMORY.md content)
`memdir.ts`	`buildMemoryPrompt(params)`	Build complete prompt with MEMORY.md content
`memdir.ts`	`loadMemoryPrompt()`	Async load and assemble complete memory prompt
`memdir.ts`	`ensureMemoryDirExists(memoryDir)`	Ensure directory exists (with guidance text)
`paths.ts`	`isAutoMemoryEnabled()`	Check global auto memory toggle
`paths.ts`	`isExtractModeActive()`	Check if extraction mode (background agent) is active
`paths.ts`	`getMemoryBaseDir()`	Get memory base directory (default ~/.claude)
`paths.ts`	`getAutoMemPath()`	Resolve auto memory directory (with memoize cache)
`paths.ts`	`getAutoMemDailyLogPath(date?)`	Get daily log path for specified date
`paths.ts`	`getAutoMemEntrypoint()`	Get MEMORY.md path
`paths.ts`	`isAutoMemPath(absolutePath)`	Check if path is within auto memory directory
`paths.ts`	`hasAutoMemPathOverride()`	Check if env var override exists
`teamMemPaths.ts`	`PathTraversalError` (class)	Path traversal detection exception
`teamMemPaths.ts`	`isTeamMemoryEnabled()`	Check if team memory is enabled
`teamMemPaths.ts`	`getTeamMemPath()`	Get team memory directory

File	Function Signature	Purpose
`teamMemPaths.ts`	`getTeamMemEntrypoint()`	Get team MEMORY.md
`teamMemPaths.ts`	`validateTeamMemWritePath(absPath)`	Validate write path security (async)
`teamMemPaths.ts`	`validateTeamMemKey(relativeKey)`	Validate server-returned relative path key
`teamMemPaths.ts`	`isTeamMemFile(filePath)`	Check if path is in team memory directory
`teamMemPrompts.ts`	`buildCombinedMemoryPrompt(extraGuidelines?, skipIndex?)`	Combine private + team memory prompt
`findRelevantMemories.ts`	`findRelevantMemories(query, memoryDir, signal, recentTools?, alreadySurfaced?)`	AI relevance retrieval main entry
`findRelevantMemories.ts`	`selectRelevantMemories(query, memories, signal, recentTools)`	Call Sonnet sideQuery to select memories

14.2 Session Memory Module (services/SessionMemory/)

File	Function/Constant	Purpose
`sessionMemory.ts`	`shouldExtractMemory(messages)`	Four-condition check for extraction trigger
`sessionMemory.ts`	`initSessionMemory()`	Initialize session memory
`sessionMemory.ts`	`manuallyExtractSessionMemory(messages, toolUseContext)`	Manually trigger extraction
`sessionMemory.ts`	`createMemoryFileCanUseTool(memoryPath)`	Build tool permission function (restrict to memory file writing)
`sessionMemory.ts`	`resetLastMemoryMessageUuid()`	Reset last memory message ID
`sessionMemoryUtils.ts`	`getLastSummarizedMessageId()`	Get last summarized message ID
`sessionMemoryUtils.ts`	`markExtractionStarted()` / `markExtractionCompleted()`	Mark extraction state
`sessionMemoryUtils.ts`	`waitForSessionMemoryExtraction()`	Wait for extraction to complete (up to 15000ms)
`sessionMemoryUtils.ts`	`getSessionMemoryContent()`	Read session memory file content
`sessionMemoryUtils.ts`	`setSessionMemoryConfig(config)`	Dynamic config update (GrowthBook remote config)
`sessionMemoryUtils.ts`	`hasMetInitializationThreshold(tokenCount)`	Check initialization threshold

File	Function/Constant	Purpose
`sessionMemoryUtils.ts`	`hasMetUpdateThreshold(tokenCount)`	Check incremental update threshold
`sessionMemoryUtils.ts`	`resetSessionMemoryState()`	Reset all global state (at session end)
`prompts.ts`	`loadSessionMemoryTemplate()`	Load template (supports file override)
`prompts.ts`	`loadSessionMemoryPrompt()`	Load extraction system prompt
`prompts.ts`	`buildSessionMemoryUpdatePrompt(currentNotes, notesPath)`	Build incremental update prompt
`prompts.ts`	`isSessionMemoryEmpty(content)`	Check if session memory is empty
`prompts.ts`	`truncateSessionMemoryForCompact(content)`	Truncate session memory during compaction

14.3 Team Memory Sync Module (services/teamMemorySync/)

File	Function/Constant	Purpose
`index.ts`	`createSyncState()`	Create initial sync state
`index.ts`	`hashContent(content)`	SHA-256 hash computation
`index.ts`	`batchDeltaByBytes(delta)`	Batch by 200KB
`index.ts`	`isTeamMemorySyncAvailable()`	Check OAuth + repo availability
`index.ts`	`pullTeamMemory(state, options?)`	Pull (supports first-time full / conditional request)
`index.ts`	`pushTeamMemory(state)`	Push (includes secret scan + delta calculation)
`index.ts`	`syncTeamMemory(state)`	Full sync (pull + push)
`watcher.ts`	`startTeamMemoryWatcher()`	Start fs.watch + debounced push
`watcher.ts`	`notifyTeamMemoryWrite()`	Trigger debounced push
`watcher.ts`	`stopTeamMemoryWatcher()`	Stop watcher
`watcher.ts`	`isPermanentFailure(result)`	Determine if permanent failure (non-409/429 4xx)
`secretScanner.ts`	`scanForSecrets(content)`	Detect sensitive info (returns ruleId + label)
`secretScanner.ts`	`getSecretLabel(ruleId)`	Get rule label (human-readable name)
`secretScanner.ts`	`redactSecrets(content)`	Sanitize (replace with `[REDACTED]`)
`types.ts`	`TeamMemoryDataSchema` (Zod)	Server response JSON schema
`types.ts`	`TeamMemoryTooManyEntriesSchema` (Zod)	413 error schema

14.4 Agent Memory Module (tools/AgentTool/)

File	Function	Purpose
`agentMemory.ts`	`getAgentMemoryDir(agentType, scope)`	Get agent memory directory
`agentMemory.ts`	`getAgentMemoryEntrypoint(agentType, scope)`	Get agent memory MEMORY.md

File	Function	Purpose
`agentMemory.ts`	`loadAgentMemoryPrompt(agentType, scope)`	Load agent memory content
`agentMemory.ts`	`isAgentMemoryPath(absolutePath)`	Check if path is in agent memory directory
`agentMemorySnapshot.ts`	`checkAgentMemorySnapshot(agentType, scope)`	Check snapshot state, decide action
`agentMemorySnapshot.ts`	`initializeFromSnapshot(agentType, scope, timestamp)`	Initialize local memory from snapshot
`agentMemorySnapshot.ts`	`replaceFromSnapshot(agentType, scope, timestamp)`	Replace local memory with snapshot
`agentMemorySnapshot.ts`	`markSnapshotSynced(agentType, scope, timestamp)`	Mark snapshot as synced

14.5 Core Utility Module (utils/)

File	Function/Constant	Purpose
`claudemd.ts`	`processMemoryFile(filePath, type, processedPaths, includeExternal, depth?, parent?)`	Parse single memory file (with @include recursion)
`claudemd.ts`	`processMdRules({ rulesDir, type, processedPaths, includeExternal, conditionalRule? })`	Process rules directory
`claudemd.ts`	`getMemoryFiles(forceIncludeExternal?)`	Main entry: get all memory files (with memoize)
`claudemd.ts`	`resetGetMemoryFilesCache(reason?)`	Reset memory files cache
`claudemd.ts`	`getClaudeMds(memoryFiles, filter?)`	Filter and concatenate to plain text
`claudemd.ts`	`getManagedAndUserConditionalRules(targetPath, processedPaths)`	Get conditional rules
`claudemd.ts`	`getExternalClaudeMdIncludes(files)`	Get external @include file list
`claudemd.ts`	`stripHtmlComments(content)`	Remove HTML comments
`claudemd.ts`	`MAX_MEMORY_CHARACTER_COUNT = 40000`	Max memory character count
`claudemd.ts`	`MAX_INCLUDE_DEPTH = 5`	@include max nesting depth
`memoryFileDetection.ts`	`detectSessionFileType(filePath)`	Detect session memory/transcript file
`memoryFileDetection.ts`	`isAutoMemFile(filePath)`	Detect auto memory file
`memoryFileDetection.ts`	`memoryScopeForPath(filePath)`	Determine personal/team scope
`memoryFileDetection.ts`	`isAutoManagedMemoryFile(filePath)`	Detect Claude-managed memory

File	Function/Constant	Purpose
		file
`memoryFileDetection.ts`	`isShellCommandTargetingMemory(command)`	Detect if shell command targets memory files

14.6 Auto-Dream Module (services/autoDream/)

File	Function/Constant	Purpose
`autoDream.ts`	`initAutoDream()`	Module initialization (create runner closure)
`autoDream.ts`	`executeAutoDream(context, appendSystemMessage?)`	Per-turn stopHook entry (non-blocking)
`autoDream.ts`	`isGateOpen()`	Check time/mode/switch three gates
`autoDream.ts`	`getConfig()`	Get GrowthBook config (tengu_onyx_plover)
`autoDream.ts`	`makeDreamProgressWatcher(taskId, setAppState)`	Listen to forked agent messages, collect file writes
`consolidationLock.ts`	`readLastConsolidatedAt()`	Read lock file mtime (i.e., lastConsolidatedAt)
`consolidationLock.ts`	`tryAcquireConsolidationLock()`	Atomically acquire lock, return priorMtime or null
`consolidationLock.ts`	`rollbackConsolidationLock(priorMtime)`	Rollback lock mtime (on fork failure)
`consolidationLock.ts`	`listSessionsTouchedSince(sinceMs)`	Get session ID list where mtime > sinceMs
`consolidationLock.ts`	`recordConsolidation()`	Record timestamp on manual /dream trigger
`consolidationPrompt.ts`	`buildConsolidationPrompt(memoryRoot, transcriptDir, extra)`	Build four-phase consolidation prompt
`config.ts`	`isAutoDreamEnabled()`	Check toggle state (settings > GrowthBook)

14.7 Memory Extraction Module (services/extractMemories/)

File	Function/Constant	Purpose
`extractMemories.ts`	`initExtractMemories()`	Module initialization (create extractor/drainer closures)
`extractMemories.ts`	`executeExtractMemories(context, appendSystemMessage?)`	Per-turn stopHook entry
`extractMemories.ts`	`drainPendingExtraction(timeoutMs?)`	Wait for all in-flight extractions before graceful shutdown
`extractMemories.ts`	`createAutoMemCanUseTool(memoryDir)`	Build tool permission function (shared with autoDream)
`extractMemories.ts`	`hasMemoryWritesSince(messages, sinceUuid)`	Detect if main agent has written memories (mutex skip)

File	Function/Constant	Purpose
`extractMemories.ts`	`countModelVisibleMessagesSince(messages, sinceUuid)`	Count new visible messages since
`extractMemories.ts`	`extractWrittenPaths(agentMessages)`	Extract Edit/Write paths from agent output
`extractMemories.ts`	`isModelVisibleMessage(message)`	Check if message is visible to model (excludes progress/system/attachment)
`prompts.ts`	`buildExtractAutoOnlyPrompt(...)`	Build private-only memory extraction prompt
`prompts.ts`	`buildExtractCombinedPrompt(...)`	Build private + team memory extraction prompt (with scope guidance)
`prompts.ts`	`opener(newMessageCount, existingMemories)`	Opening shared by both prompt variants

createAutoMemCanUseTool security design (`extractMemories.ts`):

```
export function createAutoMemCanUseTool(memoryDir: string): CanUseToolFn {
  return async (tool: Tool, input: Record<string, unknown>) => {
    // REPL mode: primitive tools are hidden, forked agent calls REPL.
    // Giving fork a different tool list would break prompt cache sharing (tool list is
cache key).
    if (tool.name === REPL_TOOL_NAME) {
      return { behavior: 'allow', updatedInput: input }
    }
    // Read tools are unrestricted
    if (tool.name === FILE_READ_TOOL_NAME || tool.name === GREP_TOOL_NAME || tool.name ===
GLOB_TOOL_NAME) {
      return { behavior: 'allow', updatedInput: input }
    }
    // Bash only allowed in read-only mode
    if (tool.name === BASH_TOOL_NAME) {
      if (tool.isReadOnly(parsed.data)) {  // tool.isReadOnly is BashTool's built-in check
        return { behavior: 'allow', updatedInput: input }
      }
      return denyAutoMemTool(tool, 'Only read-only shell commands...')
    }
    // Edit/Write only allowed for files within memoryDir
    if ((tool.name === FILE_EDIT_TOOL_NAME || tool.name === FILE_WRITE_TOOL_NAME) &&
'file_path' in input) {
      if (typeof filePath === 'string' && isAutoMemPath(filePath)) {
        return { behavior: 'allow', updatedInput: input }
      }
    }
    return denyAutoMemTool(tool, 'only Read/Grep/Glob/read-only Bash/Edit-Write-in-memdir
allowed')
  }
}
```

Design points:

- REPL mode special handling: REPL mode is enabled by default in ANT; primitive tools are hidden from direct tool list; forked agent can only call them via REPL. To prevent REPL from bypassing restrictions, inner primitive tool calls re-enter `canUseTool` check.
- Tool list as cache key (`CacheSafeParams`): giving forked agent a different tool list means it cannot reuse parent process's prompt cache, so REPL mode is special-cased rather than changing the tool list.
- `isAutoMemPath` is security-critical: only files with paths starting with `getAutoMemPath()` are allowed to write, preventing memory writes from being injected into other directories.
- `tool.isReadOnly(parsed.data)` : directly uses BashTool's built-in read-only check, no need to maintain a command whitelist.

PS:
primitive tools: Primitive tools are the lowest-level file/command operation tools. In REPL mode, while their names are hidden from the model (shown as Read/Write/Bash, etc.), the underlying implementation and rendering code still need to be aware of them:

```
FileReadTool      ← read files
FileWriteTool     ← write files
FileEditTool      ← edit files
GlobTool          ← glob file matching
GrepTool          ← content search
BashTool          ← execute commands
NotebookEditTool  ← Jupyter notebook editing
AgentTool         ← start agent
```

---

15. System Architecture Overview

15.1 Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────┐
│                        User Input                                 │
└──────────────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼
┌──────────────────────────────────────────────────────────────────┐
│              shouldShowClaudeMdExternalIncludesWarning()          │
│              Whether to warn about @include external files        │
└──────────────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼
┌──────────────────────────────────────────────────────────────────┐
│                  getMemoryFiles() (memoized)                      │
│           Load all memory files by priority (with @include recursion) │
│   Managed → User → Project → Local (later overrides earlier)      │
└──────────────────────────────────────────────────────────────────┘
                                │
                                │
             ┌──────────────────┼──────────────────┐
             ▼                  ▼                  ▼
   ┌──────────────┐    ┌──────────────┐    ┌──────────────────┐
   │  CLAUDE.md   │    │ Auto Memory  │    │  Team Memory Sync  │
   │  (4-level)   │    │ (scan+retrieval) │    │  (Watcher+Push)    │
```

```
└──────────────┘    └──────────────┘    └────────────────────┘
                          │
                          │
          ┌───────────────┼───────────────┐
          ▼               ▼               ▼

┌──────────────────┐  ┌──────────────┐  ┌──────────────────┐
│ scanMemoryFiles() │  │AI Relevance│  │ sessionMemory.ts │
│  (file scan)     │  │  Retrieval  │  │ (extraction check) │
└──────────────────┘  │findRelevant │  └──────────┬───────┘
                      │ Memories() │           │
                      └──────┬──────┘           ▼
                             │           ┌──────────────────┐
                      ┌──────▼──────┐   │ extractSessionMem │
                      │ Manifest  │   │ (AI incremental summary) │
                      │ (text)    │   └──────────┬───────┘
                      └──────┬──────┘           │
                             │                  ▼
                             ▼       ┌──────────────────────┐
                 ┌──────────────────────────────────┐
                 │        buildMemoryPrompt()            │
                 │  Assemble: type description + relevant memories + freshness │
                 └──────────────────────────────────┘
                                   │
                                   ▼
                 ┌──────────────────────────────────┐
                 │  loadMemoryPrompt() / getClaudeMds()  │
                 │  MEMORY.md truncation (200 lines/25KB) │
                 └──────────────────────────────────┘
                                   │
                                   ▼
                 ┌──────────────────────────────────┐
                 │     Main System Prompt (with Memory)   │
                 └──────────────────────────────────┘
                                   │
                  ┌────────────────┼────────────────┐
                  │                │                │
                  ▼                ▼                ▼

          ┌──────────────┐    ┌──────────────┐   ┌──────────────────┐
          │Claude Sonnet │    │    Tools     │   │  Agent Memory    │
          │  Model       │    │  (Tool Use)  │   │  (Snapshot)      │
          └──────────────┘    └──────┬───────┘   └──────────────────┘
                                     │
                        ┌────────────┼────────────┐
                        ▼            ▼            ▼

                 ┌──────────┐ ┌──────────┐   ┌──────────────────┐
                 │   Write  │ │  Git Ops │   │ Session Memory   │
                 │(Write/Edit│ │  (Bash)  │   │ Compaction       │
                 └──────────┘ └──────────┘   └──────────────────┘
                        │                            │
                        └────────────┬───────────────┘
                                     ▼

                          ┌──────────────────┐
                          │ Team Memory Watcher│
                          │  (fs.watch debounce 2s) │
```

```
                                  └──────────────────┘
                                           │
                                           ▼
                                  ┌──────────────────┐
                                  │  secretScanner   │
                                  │   (pre-upload scan) │
                                  └──────────────────┘
                                           │
                                           ▼
                                  ┌──────────────────┐
                                  │ pushTeamMemory()  │
                                  │ (delta push)      │
                                  └──────────────────┘
```

15.2 Background Process Diagram

```
========== Background Processes (independent of main conversation)============

              ┌──────────────────────────────────────────┐
              │          executeExtractMemories()        │
              │   stopHooks.ts → triggers at each turn end │
              │   Independent forked agent, writes to ~/<memory>/ │
              │   Tool constraints: read-only + write only in memoryDir │
              └──────────────────────────────────────────┘
                                  │
                                  │ Mutex check (skip if main agent already wrote)
                                  ▼
              ┌──────────────────────────────────────────┐
              │          executeAutoDream()              │
              │   stopHooks.ts → checks three gates at each turn end │
              │   Time gate ≥24h + session gate ≥5 + lock gate │
              │   Forked agent, four-phase consolidation prompt │
              │   Results written to DreamTask UI (footer pill) │
              └──────────────────────────────────────────┘
                                  │
                                  │
                        ┌─────────▼─────────┐
                        │  .consolidate-   │
                        │  lock file       │
                        │  (mtime = last-  │
                        │   ConsolidatedAt)│
                        └──────────────────┘
```

---

16. Algorithm Complexity Analysis

16.1 Operation Complexity Table

Operation	Time Complexity	Space Complexity	Notes
`scanMemoryFiles()`	O(n log n)	O(n)	n=file count, mostly from sorting
`formatMemoryManifest()`	O(n)	O(n)	manifest string construction
`findRelevantMemories()`	O(n) + O(1) API	O(n) manifest	AI call is side query, not counted in main complexity
`syncTeamMemory()`	O(n) hash + O(k) push	O(1) incremental	k=changed files, k≤n

Operation	Time Complexity	Space Complexity	Notes
`shouldExtractMemory()`	O(1)	O(1)	condition check only
`@include` parsing	O(d), d≤5	O(d) call stack	depth-bounded, d≤5
`truncateEntrypointContent()`	O(L+B)	O(1)	L=lines, B=bytes
`calculateMessagesToKeepIndex()`	O(m)	O(1)	m=message count, linear scan
`processMemoryFile()`	O(f × d)	O(d)	f=included files, d=depth≤5

---

17. Auto-Dream System (Automatic Memory Consolidation)

17.1 System Positioning

Auto-Dream is Claude Code's **background memory consolidation mechanism**, running a dedicated forked subagent in the background to deeply organize accumulated session memories. It is not a user-initiated command, but an automatically triggered background process that activates when specific conditions are met.

Relationship with main agent memory writing:

- Main agent writes memories during conversation based on system prompt instructions (per-session)
- extractMemories does lightweight supplemental extraction at each turn end (also forked agent)
- Auto-Dream is a periodic **deep consolidation**, responsible for **merging, deduplication, correction, and index updating**

Three-gate trigger mechanism (Gate Order, cheapest checks first):

```
Trigger condition check order:

Gate 1 — Time gate (inside isGateOpen)
  → ≥ minHours since last consolidation (default 24h)
  → Checked per turn: one stat call

Gate 2 — Session gate (inside isGateOpen)
  → ≥ minSessions sessions with changes since last consolidation (default 5)
  → Scan throttled: skip if last scan was < SESSION_SCAN_INTERVAL_MS (10min) ago

Gate 3 — Lock gate
  → .consolidate-lock file mtime = lastConsolidatedAt
  → PID check + 60min expiration protection, prevents zombie processes
```

PS: Gate check order is sorted by computational cost from low to high. Time gate costs only one stat; if skipped, check session gate; session gate has slightly higher computation but is throttled at 10 minutes; lock gate is heaviest but already the last step before triggering

17.2 Lock File Mechanism (consolidationLock.ts)

Lock file path: `~/.claude/projects/<sanitized-git-root>/memory/.consolidate-lock`

Design points:

- Lock file is inside memory directory, follows git root; different worktrees don't interfere with each other
- Lock file body = PID, mtime = lastConsolidatedAt (so mtime doubles as timestamp)

- Holder expiration time (HOLDER_STALE_MS) = 60 minutes, prevents PID reuse causing permanent deadlock
- rollbackConsolidationLock: on fork failure, rollback mtime so time gate can pass next time

```
// Atomic lock acquisition flow
tryAcquireConsolidationLock():
  1. Read lock file mtime + PID
  2. If mtime < 60min and PID alive → lock held, return null
  3. Write current PID (mtime = now)
  4. Re-read to verify (prevent race), PID mismatch returns null
  5. Return priorMtime (for rollback on failure)
```

17.3 Four-Phase Consolidation Prompt (consolidationPrompt.ts)

The consolidation prompt guides the forked agent through four phases:

```
Phase 1 — Orient
  ls memory directory + read MEMORY.md to understand current index
  Avoid duplication: first look at existing topic files, then decide what to write

Phase 2 — Gather recent signal
  Check sources by priority:
    1. Daily logs (logs/YYYY/MM/YYYY-MM-DD.md) — daily append logs
    2. Outdated memories (facts that contradict current code)
    3. Transcript search (grep JSONL, exact match only, not full read)
  Principle: don't exhaustively read transcript, only find things already suspected
important

Phase 3 — Consolidate
  For each piece worth remembering:
    → Merge into existing topic file (rather than creating near-duplicates)
    → Relative dates → absolute dates ("yesterday" → "2026-04-07")
    → Delete discredited facts

Phase 4 — Prune and index
  Update MEMORY.md (≤200 lines, ≤~25KB):
    → Delete pointers to outdated/wrong/replaced memories
    → Shorten overly long entries (>200 chars should be shortened, move details to topic
file)
    → Add pointers to new important memories
    → Resolve contradictions (when two files disagree, fix the wrong one)
```

17.4 Tool Constraints

Auto-Dream forked agent tool constraints (from `createAutoMemCanUseTool`):

- Allowed: Read / Grep / Glob (unrestricted)
- Allowed: Read-only Bash (`ls`, `find`, `grep`, `cat`, `stat`, `wc`, `head`, `tail`, etc.)
- Allowed: Edit / Write **only within memory directory**
- Denied: Write-capable Bash (like `>` redirection, `|` pipe to file), MCP tools, Agent tools

PS: These constraints are identical to extractMemories, shared via `createAutoMemCanUseTool()`

17.5 DreamTask UI Integration

Auto-Dream's forked agent runs in the background, invisible to users — DreamTask "visualizes" it in the footer pill and Shift+Down floating panel.

```
// DreamTaskState structure
{
  type: 'dream',
  phase: 'starting' | 'updating',  // switches from starting to updating after Edit/Write
received
  sessionsReviewing: number,        // number of sessions to review
  filesTouched: string[],           // observed file paths (from tool_use, incomplete)
  turns: DreamTurn[],               // assistant text responses, tool calls collapsed to
count
  abortController?: AbortController,
  priorMtime: number,              // used for lock rollback on kill
}

// kill() behavior:
// 1. abort abortController (abort forked agent)
// 2. status → killed
// 3. rollback lock mtime (rollbackConsolidationLock)
```

PS: filesTouched is incomplete — only captures Edit/Write tool_use, not bash-mediated writes, so it means "at least these were changed," not "only these were changed"

17.6 GrowthBook Configuration and Toggle

```
// autoDream.ts — getConfig() reads GrowthBook
' tengu_onyx_plover': {
  minHours: 24,       // default 24 hours
  minSessions: 5,     // default 5 sessions
}

// config.ts — isAutoDreamEnabled() priority
1. settings.json's autoDreamEnabled (user explicit setting)
2. GrowthBook 'tengu_onyx_plover'.enabled (experiment group)
3. Return false
```

17.7 Relationship with /dream Skill

The `/dream` skill (`skills/bundled/dream.ts`) is a manually triggered dream command, behind `KAIROS` or `KAIROS_DREAM` feature flag:

- On manual trigger, calls `recordConsolidation()` to optimistically update timestamp
- Also uses the same consolidation prompt
- Also a forked agent, but goes through /dream skill path, bypassing autoDream's three-gate check

---

18. ExtractMemories System (End-of-Session Memory Extraction)

18.1 System Positioning

ExtractMemories is a lightweight memory extraction triggered at the end of each conversation turn (`stopHooks`), running via **forked agent**. It is not a replacement for the main agent writing its own memories, but a complementary mechanism — when the main agent hasn't actively written memories, it automatically mines extractable information.

18.2 Forked Agent Mode

Extraction uses `runForkedAgent()` , which is a perfect fork of the main conversation:

- Same system prompt (includes complete memory-saving instructions)
- Shares parent process's prompt cache
- Runs independently, doesn't write back to main transcript (`skipTranscript: true`)
- Max 5 turns (`maxTurns: 5`) to prevent verification rabbit-holes

Mutex mechanism (hasMemoryWritesSince):
If the main agent has already written memory files during this turn (detected by checking if tool_use contains Edit/Write to autoMemPath), extraction is skipped and the cursor advances directly.

```
// Mutex: main agent already wrote → skip extraction
if (hasMemoryWritesSince(messages, lastMemoryMessageUuid)) {
  lastMemoryMessageUuid = lastMessage.uuid  // advance cursor, skip processing
  return
}
```

18.3 Closure State and Trailing Run

The extraction module creates a closure via `initExtractMemories()` , capturing the following state:

```
let lastMemoryMessageUuid        // cursor: UUID of last processed message
let inProgress = false           // is extraction currently running
let turnsSinceLastExtraction = 0 // throttle counter
let pendingContext               // context accumulated during run (for trailing run)
let inFlightExtractions = Set    // all pending extraction Promises
```

Trailing Run mechanism:

- If another extraction is triggered during a running extraction → accumulate to `pendingContext`
- After current extraction completes, check `pendingContext` and execute trailing run (skipping throttle)
- Ensures new messages aren't missed

PS:

- "Closure state" refers to: after the main conversation ends, the background memory extraction task **still holds references to the state at conversation end** (e.g., conversation context, pending operations, etc.). This state should be cleaned up when the session closes, but because the extraction task is still running, it gets "enclosed" in the closure
- Essentially: **background tasks hold references to the context of a ended session, preventing it from being garbage collected**

18.4 Submission Pipeline

```
executeExtractMemories() triggers
  → Check GrowthBook gate (tengu_passport_quail)
  → Check isAutoMemoryEnabled()
  → Check inProgress (decide whether to stash trailing)
  → Call scanMemoryFiles() to get manifest (pre-inject, avoid agent wasting a turn on ls)
  → Build extraction prompt (with newMessageCount + existingMemories)

  → runForkedAgent({
      canUseTool: createAutoMemCanUseTool(memoryDir),
      maxTurns: 5,
      skipTranscript: true,
    })
  → Extract writtenPaths
  → appendSystemMessage({ "Saved N memories" }) to main conversation
  → Refresh cursor lastMemoryMessageUuid

drainPendingExtraction() — called on graceful shutdown
  → Wait for all inFlightExtractions to complete (up to 60s)
  → Prevents killing running forked agent during shutdown
```

18.5 Extraction Prompt Templates

Two prompt variants, controlled by `tengu_moth_copse` GrowthBook flag for whether to skip index writing:

Auto-Only variant (`buildExtractAutoOnlyPrompt`):

- Extracts **private personal memories only**, stored in local directory
- Tells main agent: if user asks to remember something, save immediately
- Provides four type definitions (user/feedback/project/reference)
- Provides "What NOT to save" list
- Two-step save: ① write topic file ② update MEMORY.md (index)

Combined variant (`buildExtractCombinedPrompt` , team-only feature):

- Adds **team memory** on top of Auto-Only
- Additional `<scope>` tag for each type (private vs team)
- Additional requirement: don't save sensitive data in team memory
- Two-step save: do separately for private and team directories

---

19. DreamTask (Dream Task UI Integration)

19.1 Design Background

Auto-Dream's forked agent is invisible to users by default. DreamTask integrates it into the UI via the task registry framework (`registerTask` / `updateTaskState`), making the background process visible in the footer pill and Shift+Down dialog.

19.2 Task State Structure

```typescript
type DreamTaskState = {
  type: 'dream'            // task type identifier
  status: 'running' | 'completed' | 'failed' | 'killed'
```

```
  phase: 'starting' | 'updating'
  sessionsReviewing: number
  filesTouched: string[]  // at least modified file paths (incomplete)
  turns: DreamTurn[]      // last 30 assistant responses
  priorMtime: number      // used for lock rollback on kill
}
```

19.3 Message Listening (makeDreamProgressWatcher)

`makeDreamProgressWatcher` in autoDream.ts listens to each assistant message from the forked agent: Whenever the forked agent sends an assistant message, immediately extract three things:

- Collect text blocks into turn.text
- Count tool_use block quantity
- Extract file_path from Edit/Write tool_use into filesTouched
  PS: Because Auto Dream is a background fork process, users can't see its full output, but it edits files. Here the operation creates a structured summary informing the user "what Auto Dream did" — collects progress in real time, summarizes for display upon completion.

19.4 Kill Task

Terminating Dream from UI:

1. `abortController.abort()` — abort API call
2. `rollbackConsolidationLock(priorMtime)` — rollback lock mtime, enabling retry next time

---

20. Daily Log Mode (Assistant Mode / KAIROS)

20.1 Background

In Assistant mode (perpetual session, as opposed to CLI mode running separately each time), **conversations are continuous and cannot rely on conversation stopping points**. Claude Code designed the **Daily Log** mode for this.

20.2 Differences from Standard Mode

Feature	Standard Mode (CLI)	Daily Log Mode (Assistant)
New memory destination	Write directly to topic file	Append to `logs/YYYY/MM/YYYY-MM-DD.md`
MEMORY.md maintenance	Update index after each memory write	暂时不维护 (log is append-only)
Index consolidation	None	由 nightly /dream skill periodically distills logs into topic files
Trigger condition	—	feature('KAIROS')

PS:

- Non-KAIROS mode: uses Auto Dream (forked agent background memory consolidation)
- KAIROS mode: uses disk-skill dream, Auto Dream is skipped

---

21. Startup Chain and Module Initialization

21.1 backgroundHousekeeping.ts

All memory modules are initialized in `backgroundHousekeeping.ts` (in parallel with `initExtractMemories`):

```
export function startBackgroundHousekeeping(): void {
  // ① Always initialize (unconditional)
  void initMagicDocs()           // Magic Docs service
  void initSkillImprovement()    // Skill improvement

  // ② Conditional compilation (if EXTRACT_MEMORIES not enabled, tree-shaken)
  if (feature('EXTRACT_MEMORIES')) {
    extractMemoriesModule!.initExtractMemories()
  }

  // ③ Always initialize (after extractMemories)
  initAutoDream()

  // ④ Plugin auto-update
  void autoUpdateMarketplacesAndPluginsInBackground()

  // ⑤ Deep link registration (interactive mode + LODESTONE only)
  if (feature('LODESTONE') && getIsInteractive()) {
    void registerProtocolModule!.ensureDeepLinkProtocolRegistered()
  }

  // ⑥ Delay slow operations (execute after 10 minutes)
  setTimeout(runVerySlowOps, 10min).unref()

  // ⑦ ant users: periodic cleanup every 24 hours
  if (process.env.USER_TYPE === 'ant') {
    setInterval(cleanupNpmCache + cleanupOldVersions, 24h).unref()
  }
}
```

backgroundHousekeeping.ts is the **background task housekeeper** launched in parallel at startup: initializes memory/docs/Dream services, schedules cleanup tasks, and does so without blocking main process startup.

PS:

- Delayed slow operations (runVerySlowOps): during startup, if the user is already operating, cleanup tasks should not compete for resources. Wait until user is idle.
- .unref() — does not block process exit: these timers don't count toward Node.js's "active references." Process exit won't be blocked by these timers.

21.2 stopHooks.ts Trigger Chain

```
// src/query/stopHooks.ts
// Triggers at end of each conversation turn (model outputs final response, no tool call):
void executeExtractMemories(stopHookContext, appendSystemMessage)
void executeAutoDream(stopHookContext, appendSystemMessage)
```

Both systems are fire-and-forget (do not await completion), so they run in parallel and non-blocking.

21.3 Drain on Shutdown

```
// print.ts — after response flush, before gracefulShutdownSync
await drainPendingExtraction(60_000)  // wait up to 60s
```

Appendix A: Zod Schema Definitions

A.1 Schema Definitions

```
// teamMemorySync/types.ts
const TeamMemoryContentSchema = z.object({
  entries: z.record(z.string(), z.string()),           // key -> content
  entryChecksums: z.record(z.string(), z.string()).optional(),
})

const TeamMemoryDataSchema = z.object({
  organizationId: z.string(),
  repo: z.string(),
  version: z.number(),
  lastModified: z.string(),
  checksum: z.string(),
  content: TeamMemoryContentSchema(),
})

// agentMemorySnapshot.ts
const snapshotMetaSchema = z.object({
  updatedAt: z.string().min(1),
})

const syncedMetaSchema = z.object({
  syncedFrom: z.string().min(1),
})
```

Appendix B: Global State Overview

B.1 Global State per Module

Session Memory module maintains the following global state (`sessionMemoryUtils.ts`):

```
let lastSummarizedMessageId: string | undefined = undefined
let extractionStartedAt: number | undefined = undefined
let tokensAtLastExtraction = 0
let sessionMemoryInitialized = false
let sessionMemoryConfig: SessionMemoryConfig = { ...DEFAULT_SESSION_MEMORY_CONFIG }
```

Watch module maintains the following global state (`watcher.ts`):

```
let watcher: FSWatcher | null = null
let debounceTimer: ReturnType<typeof setTimeout> | null = null
let pushInProgress = false
```

```
let hasPendingChanges = false
let currentPushPromise: Promise<void> | null = null
let watcherStarted = false
let pushSuppressedReason: string | null = null
let syncState: SyncState | null = null
```

ExtractMemories module maintains the following global state (`extractMemories.ts` , closure scope):

```
let lastMemoryMessageUuid: string | undefined        // cursor: last processed message UUID
let inProgress = false                               // is extraction currently running
let turnsSinceLastExtraction = 0                    // throttle counter
let pendingContext: { context, appendSystemMessage } | undefined  // trailing run
accumulated context
let inFlightExtractions = new Set<Promise<void>>()  // all pending extraction Promises
let hasLoggedGateFailure = false                    // whether gate failure has been logged
```

AutoDream module maintains the following global state (`autoDream.ts` , closure scope):

```
let runner: ((context, appendSystemMessage?) => Promise<void>) | null = null
let lastSessionScanAt = 0  // last scan time (for SESSION_SCAN_INTERVAL_MS throttling)
```